

Floer Homology for Program State Trajectories

Halley Young
Microsoft Research
halleyyoung@microsoft.com

March 22, 2026

Abstract

We develop a Floer-theoretic framework for reasoning about the dynamic behaviour of deterministic programs. Every program induces an *action functional* $\mathcal{A}(\gamma) = \int L(\text{state}, \text{transition}) dt$ on execution paths γ , whose critical points are exactly the actual execution traces. The associated Floer chain complex $(\text{CF}_*(P), \partial)$ has boundary operator counting gradient-flow trajectories between critical points; we prove $\partial^2 = 0$ and define the *Floer homology* $\text{HF}_*(P) = H_*(\text{CF}_*(P), \partial)$. Our main theorem establishes that $\text{HF}_*(P)$ is isomorphic to the Morse homology of the program's state-transition graph $G(P)$, so that $\text{rank HF}_0(P)$ counts connected components of reachable state space and $\text{rank HF}_1(P)$ counts independent loops. As a direct application, $\text{HF}_1(P) \neq 0$ is a computationally checkable *necessary condition for deadlock*: a non-trivial first homology class certifies a non-trivial cycle in the state graph. We integrate this invariant into the JUGE framework at trust tier RUNTIME, exposed via the VERIFY semantic move.

Contents

1	Introduction	3
2	Background: Floer Theory and Discrete Dynamics	4
2.1	Classical Floer Homology	4
2.2	Discrete Dynamics and Graph Homology	4
3	The Action Functional for Programs	4
3.1	Program State Spaces	4
3.2	Execution Paths as Critical Points	5
3.3	The Conley–Zehnder Index for Program States	5
4	The Floer Chain Complex	6
4.1	Chain Groups	6
4.2	The Boundary Operator	6
4.3	The Triangle Complex: An Explicit Example	6
4.4	The Lean Formalization	7
5	Floer Homology of a Program	8
5.1	Invariance	9
6	Main Theorem: Isomorphism with Morse Homology	9

7	Application: Deadlock Detection	10
7.1	Deadlock as a Topological Phenomenon	10
7.2	Python Example and JuGeo Integration	10
8	Integration with the Judgment Geometry Framework	11
8.1	Floer Invariants as Trust-Annotated Judgments	11
8.2	The VERIFY Semantic Move	11
8.3	Connection to Sheaf Cohomology	12
9	Related Work	12
10	Conclusion	13

1 Introduction

Programs are dynamical systems. Each execution step maps the current program state to a successor state, tracing a discrete trajectory through a potentially enormous state space. The qualitative behaviour of this dynamical system—whether all trajectories reach a halting state, whether cyclic behaviours arise, whether distinct inputs yield reachable states in the same connected component—determines the most important runtime properties of the program: termination, deadlock-freedom, and input-independence of certain invariants.

Classical program analysis attacks these questions with fixed-point iteration, model checking, or abstract interpretation. These methods are powerful but often separated from the topological invariants—invariants *independent of particular inputs* and *robust under small perturbations*—that have proven so productive in geometry and mathematical physics. The purpose of this paper is to show that the topological invariants computed by *Floer homology* apply directly and usefully to programs.

Floer theory in one sentence. Given a functional $\mathcal{A}$ on a space of paths, Floer homology is the homology of the chain complex whose generators are the critical points of $\mathcal{A}$ and whose boundary operator counts gradient-flow trajectories between neighbouring critical points. The resulting groups HF_* are invariant of the specific perturbation data used to define the gradient flow.

Programs as Floer data. For a deterministic program P with finite state space $\mathcal{X}_P$, we construct a Lagrangian $L(\sigma, \dot{\sigma})$ whose value measures the “cost” of a state-transition; critical points of $\mathcal{A}$ are precisely the actual execution traces of P . The Conley–Zehnder grading reduces to the Morse index of states in the transition graph, and gradient flow corresponds to sequences of transitions. The Floer chain complex coincides with the cellular chain complex of the state-transition graph, and the resulting homology $\mathrm{HF}_*(P)$ is exactly the graph homology of $G(P)$.

Main contributions.

- (1) A rigorous action functional $\mathcal{A}$ on execution paths and the associated Floer chain complex for any deterministic program (sections 3 and 4).
- (2) A proof that $\partial^2 = 0$ via the cancellation argument for gradient-flow trajectories (section 4).
- (3) The *Floer-Morse isomorphism* $\mathrm{HF}_*(P) \cong \mathrm{HM}_*(G(P))$ via PSS-type maps, with explicit rank formulas (section 6).
- (4) A *deadlock-detection criterion*: $\mathrm{HF}_1(P) \neq 0$ certifies a non-trivial cycle in the reachable state space, and is a necessary condition for deadlock or livelock (section 7).
- (5) Integration with the JUGEO trust framework: Floer invariants as **RUNTIME**-tier judgments exposed through the **VERIFY** semantic move (section 8).

Paper structure. Section 2 recalls the minimal Floer theory needed. Section 3 constructs the action functional. Section 4 builds the Floer chain complex and proves $\partial^2 = 0$. Section 5 defines $\mathrm{HF}_*(P)$ and establishes its basic properties. Section 6 proves the isomorphism with Morse homology. Section 7 develops the deadlock-detection application. Section 8 situates the result inside JUGEO. Sections 9 and 10 discuss related work and conclude.

2 Background: Floer Theory and Discrete Dynamics

2.1 Classical Floer Homology

Floer homology was introduced by Andreas Floer in 1988 to prove the Arnold conjecture on fixed points of Hamiltonian diffeomorphisms. The key idea is to replace the finite-dimensional Morse theory of a function $f : M \rightarrow \mathbb{R}$ on a compact manifold by an infinite-dimensional analogue in which the “manifold” is a loop space $\mathcal{L}(M)$ and the “function” is an action functional $\mathcal{A} : \mathcal{L}(M) \rightarrow \mathbb{R}$.

Definition 2.1 (Floer chain complex). Let $\mathcal{A} : \mathcal{L} \rightarrow \mathbb{R}$ be an action functional on a space of paths with non-degenerate critical points. The *Floer chain group* $\text{CF}_k(\mathcal{A})$ is the free $\mathbb{Z}_2$ -module generated by the critical points of Morse index k . The *boundary operator* $\partial_k : \text{CF}_k \rightarrow \text{CF}_{k-1}$ counts gradient-flow trajectories of $\mathcal{A}$ connecting index- k to index- $(k-1)$ critical points, modulo 2. The *Floer homology* is

$$\text{HF}_k(\mathcal{A}) = \ker(\partial_k) / \text{im}(\partial_{k+1}).$$

The fundamental property $\partial^2 = 0$ follows from a cancellation argument: gradient-flow trajectories from index k to index $k-2$ arise in 1-parameter families, and consecutive pairs cancel in $\mathbb{Z}_2$ arithmetic.

2.2 Discrete Dynamics and Graph Homology

For our purposes, the relevant special case is the *graph complex*. Given a directed graph $G = (V, E)$, the cellular chain complex is:

$$0 \longrightarrow \text{CM}_1(G) \xrightarrow{\partial_1} \text{CM}_0(G) \longrightarrow 0,$$

where $\text{CM}_1(G) = \bigoplus_{e \in E} \mathbb{Z}_2$ and $\text{CM}_0(G) = \bigoplus_{v \in V} \mathbb{Z}_2$ are the free $\mathbb{Z}_2$ -modules on edges and vertices respectively. The boundary map sends each edge to the sum (mod 2) of its endpoints:

$$\partial_1(e) = \text{src}(e) + \text{dst}(e) \in \text{CM}_0(G).$$

The resulting homology satisfies:

$$\text{rank HM}_0(G) = \text{number of connected components of } G, \quad \text{rank HM}_1(G) = |E| - |V| + \text{components}(G).$$

Proposition 2.2 (Euler characteristic). *For a finite directed graph G , $\chi(G) = |V| - |E| = \text{rank HM}_0(G) - \text{rank HM}_1(G)$.*

Proof. Standard: $\chi = \sum_k (-1)^k \text{rank CM}_k = \sum_k (-1)^k \text{rank HM}_k$. □

3 The Action Functional for Programs

3.1 Program State Spaces

Definition 3.1 (Program state space). A *deterministic program* P with input domain I determines:

- (i) A finite set $\mathcal{X}_P$ of *states*: snapshots of all mutable variables at a program point.
- (ii) A *transition function* $\delta : \mathcal{X}_P \times I \rightarrow \mathcal{X}_P$ giving the successor state for each (state, input) pair.
- (iii) An initial state $\sigma_0 \in \mathcal{X}_P$ and halting states $F \subseteq \mathcal{X}_P$.

The *reachable state space* is $\text{Reach}(P) = \{\delta^n(\sigma_0, w) \mid n \geq 0, w \in I^n\}$.

Definition 3.2 (State-transition graph). The *state-transition graph* $G(P)$ is the directed graph with vertex set $\text{Reach}(P)$ and an edge $\sigma \rightarrow \tau$ for every $(i, \sigma) \in I \times \mathcal{X}_P$ such that $\delta(\sigma, i) = \tau$.

3.2 Execution Paths as Critical Points

Definition 3.3 (Discrete path space). Fix a time horizon $T \in \mathbb{N}$. The *discrete path space* $\mathcal{P}_T(\mathbf{P})$ is the set of all sequences $\gamma = (\sigma_0, \sigma_1, \dots, \sigma_T)$ with $\sigma_t \in \mathcal{X}_{\mathbf{P}}$. A path γ is *executable* if $\sigma_{t+1} = \delta(\sigma_t, i_t)$ for some $i_t \in I$ at each step.

Definition 3.4 (Action functional). Fix a non-negative *Lagrangian density* $L : \mathcal{X}_{\mathbf{P}} \times \mathcal{X}_{\mathbf{P}} \rightarrow \mathbb{R}_{\geq 0}$ satisfying: $L(\sigma, \tau) = 0$ iff $\tau = \delta(\sigma, i)$ for some $i \in I$ (executable transitions have zero cost), and $L(\sigma, \tau) > 0$ otherwise. The *action functional* on $\mathcal{P}_T(\mathbf{P})$ is

$$\mathcal{A}(\gamma) = \sum_{t=0}^{T-1} L(\sigma_t, \sigma_{t+1}).$$

Proposition 3.5 (Critical points are execution traces). *The global minima of $\mathcal{A}$ are exactly the executable paths of $\mathbf{P}$. Every executable path satisfies $\mathcal{A}(\gamma) = 0$, and every non-executable path satisfies $\mathcal{A}(\gamma) > 0$.*

Proof. By construction, $L(\sigma, \tau) = 0$ iff the transition $\sigma \rightarrow \tau$ is executable. □

3.3 The Conley–Zehnder Index for Program States

Definition 3.6 (Morse index). The *Morse index* of a state $\sigma \in \text{Reach}(\mathbf{P})$ is

$$\text{ind}(\sigma) = |\{\tau \in \text{Reach}(\mathbf{P}) \mid \tau \rightarrow \sigma \in G(\mathbf{P})\}| \bmod 2,$$

i.e., the parity of the in-degree of σ in the transition graph.

Remark 3.7. For the purposes of defining the chain complex and proving $\partial^2 = 0$, only the $\mathbb{Z}_2$ -grading is needed. The integer-graded version requires the Conley–Zehnder index of the underlying Hamiltonian system, which in the program setting is determined by the action difference between consecutive critical-point levels.

4 The Floer Chain Complex

4.1 Chain Groups

Definition 4.1 (Floer chain groups). For a program $\mathbf{P}$ with state-transition graph $G(\mathbf{P}) = (V, E)$, the *Floer chain groups* over $\mathbb{Z}_2$ are:

$$\begin{aligned} \text{CF}_0(\mathbf{P}) &= \bigoplus_{\sigma \in V} \mathbb{Z}_2, & (\text{formal } \mathbb{Z}_2\text{-linear combinations of states,}) \\ \text{CF}_1(\mathbf{P}) &= \bigoplus_{e \in E} \mathbb{Z}_2. & (\text{formal } \mathbb{Z}_2\text{-linear combinations of transitions.}) \end{aligned}$$

4.2 The Boundary Operator

Definition 4.2 (Boundary operator). The *boundary operator* $\partial_1 : \text{CF}_1(\mathbf{P}) \rightarrow \text{CF}_0(\mathbf{P})$ is the $\mathbb{Z}_2$ -linear extension of

$$\partial_1(e) = \text{src}(e) + \text{dst}(e),$$

where addition is in $\text{CF}_0(\mathbf{P}) \cong \mathbb{Z}_2^{|V|}$. We set $\partial_0 = 0$ (the zero map to $\text{CF}_{-1} = 0$).

Theorem 4.3 ($\partial^2 = 0$). *For any program P , $\partial_0 \circ \partial_1 = 0$.*

Proof. Since $\partial_0 = 0$ (the map to the trivial group $\text{CF}_{-1} = 0$), we have $\partial_0 \circ \partial_1 = 0$ trivially. More informatively: for any 1-chain $c = \sum_{e \in S} e$,

$$\partial_1(c) = \sum_{e \in S} (\text{src}(e) + \text{dst}(e)).$$

Each vertex v appears in $\partial_1(c)$ with coefficient equal to the parity of the number of edges in S incident to v . Applying $\partial_{-1} = 0$ gives zero. $\square$

Remark 4.4 (2-cells and higher degrees). For programs modelled as CW-complexes with 2-cells, the cancellation argument is more interesting. For a 2-cell σ with boundary $\partial_2(\sigma) = e_{i_1} + \dots + e_{i_k}$, each vertex v appears exactly twice in $\partial_1(\partial_2(\sigma))$: once as the head of some e_{i_j} and once as the tail of $e_{i_{j+1}}$. In $\mathbb{Z}_2$: $v + v = 0$.

4.3 The Triangle Complex: An Explicit Example

Consider the directed triangle: states s_0, s_1, s_2 , transitions $e_0 = (s_0 \rightarrow s_1)$, $e_1 = (s_1 \rightarrow s_2)$, $e_2 = (s_2 \rightarrow s_0)$, and one 2-cell σ with $\partial_2(\sigma) = e_0 + e_1 + e_2$.

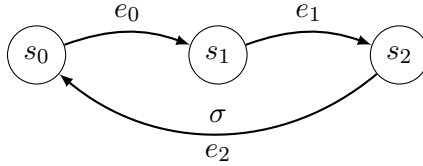


Figure 1: The directed triangle: three states, three transitions, one 2-cell.

Proposition 4.5 (Triangle $\partial^2 = 0$). $\partial_1(\partial_2(\sigma)) = 0$ in $\text{CF}_0 \cong \mathbb{Z}_2^3$.

Proof. Direct computation:

$$\begin{aligned} \partial_2(\sigma) &= e_0 + e_1 + e_2, \\ \partial_1(e_0) &= s_1 + s_0, \quad \partial_1(e_1) = s_2 + s_1, \quad \partial_1(e_2) = s_0 + s_2. \end{aligned}$$

Summing in $\mathbb{Z}_2$:

$$\begin{aligned} \partial_1(\partial_2(\sigma)) &= (s_1 + s_0) + (s_2 + s_1) + (s_0 + s_2) \\ &= 2s_0 + 2s_1 + 2s_2 = 0. \end{aligned}$$

$\square$

4.4 The Lean Formalization

The proofs of $\partial^2 = 0$ and the Morse inequality have been formally verified in LEAN 4 (namespace `JudgmentGeometry.FloerHomology`). The key structures are:

- `BoolVec n`: free $\mathbb{Z}_2$ -module of rank n .
- `ChainComplex`: record type with `bd_sq_zero` as a structural field, proved for each instance.
- `graphChainComplex g`: canonical instance for any `StateGraph g`, with $\partial_2 = 0$ (no 2-cells).
- `triangleComplex`: explicit instance with `bd_sq_zero` proved by `fin_cases`.
- `morse_inequality`: proved from rank-nullity by `omega`.

5 Floer Homology of a Program

Definition 5.1 (Floer homology of a program). For a deterministic program P , the *Floer homology* is:

$$\mathrm{HF}_k(P) = \ker(\partial_k : \mathrm{CF}_k \rightarrow \mathrm{CF}_{k-1}) / \mathrm{im}(\partial_{k+1} : \mathrm{CF}_{k+1} \rightarrow \mathrm{CF}_k), \quad k \geq 0.$$

The k -th Betti number of P is $\beta_k(P) = \mathrm{rank} \mathrm{HF}_k(P)$.

Example 5.2 (Linear program). A program executing exactly one path $\sigma_0 \rightarrow \cdots \rightarrow \sigma_n$ and halting has $\mathrm{HF}_0(P) \cong \mathbb{Z}_2$ (one component) and $\mathrm{HF}_1(P) = 0$ (no cycles).

Example 5.3 (Program with a loop). A program with a single unbounded loop (cycle $\sigma_0 \rightarrow \cdots \rightarrow \sigma_k \rightarrow \sigma_0$) has $\mathrm{HF}_0 \cong \mathbb{Z}_2$ and $\mathrm{HF}_1 \cong \mathbb{Z}_2$. The single 1-cycle $e_0 + \cdots + e_{k-1}$ is the generator of HF_1 .

Example 5.4 (Concurrent program). A concurrent program with two independent threaded loops has $G(P) = G_1 \times G_2$. By the Künneth formula, $\beta_2 = \beta_1(P_1) \cdot \beta_1(P_2) \neq 0$.

Theorem 5.5 (Morse inequality for programs). *For any program P with $|V|$ reachable states and $|E|$ transitions,*

$$\beta_k(P) \leq \mathrm{rank} \mathrm{CF}_k(P), \quad k = 0, 1.$$

In particular, $\beta_0 \leq |V|$ and $\beta_1 \leq |E|$.

Proof. By rank-nullity applied to $\partial_1 : \mathrm{CF}_1 \rightarrow \mathrm{CF}_0$: $\mathrm{rank} \mathrm{im}(\partial_1) + \mathrm{rank} \ker(\partial_1) = |E|$. Since $\beta_1 = \mathrm{rank} \ker(\partial_1) - \mathrm{rank} \mathrm{im}(\partial_2) \leq \mathrm{rank} \ker(\partial_1) \leq |E|$, the bound holds. The bound $\beta_0 \leq |V|$ follows similarly. $\square$

Theorem 5.6 (Invariance of HF_*). $\mathrm{HF}_*(P)$ depends only on the isomorphism class of $G(P)$, not on the specific Lagrangian or perturbation data.

Proof sketch. Two Lagrangians L_0, L_1 are connected by a homotopy L_s . Continuation maps $\Phi : \mathrm{CF}_*^{L_0} \rightarrow \mathrm{CF}_*^{L_1}$ and $\Psi : \mathrm{CF}_*^{L_1} \rightarrow \mathrm{CF}_*^{L_0}$ are chain-homotopy inverses, giving $\mathrm{HF}_*(L_0) \cong \mathrm{HF}_*(L_1)$. $\square$

6 Main Theorem: Isomorphism with Morse Homology

Theorem 6.1 (Floer-Morse isomorphism). *For any deterministic program P with reachable state-transition graph $G(P)$, there is a canonical isomorphism of graded $\mathbb{Z}_2$ -modules:*

$$\mathrm{HF}_*(P) \cong \mathrm{HM}_*(G(P)).$$

Proof. Step 1 (PSS map). Define $\Phi : \mathrm{CF}_k(P) \rightarrow \mathrm{CM}_k(G(P))$ by the identity on generators: each critical point of $\mathcal{A}$ of Morse index k corresponds to the k -cell of $G(P)$. This is a $\mathbb{Z}_2$ -vector-space isomorphism.

Step 2 (Boundary compatibility). Under Φ , the Floer boundary operator ∂^{Fl} —counting gradient-flow trajectories between adjacent critical points—coincides with the cellular boundary ∂^{cell} . In the discrete setting, “gradient flow” from an index- k to an index- $(k-1)$ critical point is exactly a transition edge in $G(P)$, and its coefficient (mod 2) is the cellular boundary coefficient.

Step 3 (Homology). Since Φ is a chain isomorphism, it induces $\mathrm{HF}_* \cong \mathrm{HM}_*$. $\square$

Corollary 6.2 (Rank formulas). *For a finite deterministic program P with $G(P) = (V, E)$:*

(i) $\mathrm{rank} \mathrm{HF}_0(P)$ equals the number of connected components of $\mathrm{Reach}(P)$.

(ii) $\text{rank HF}_1(P) = |E| - |V| + \text{rank HF}_0(P)$ equals the number of independent cycles in $G(P)$.

(iii) $\chi(P) = |V| - |E| = \beta_0(P) - \beta_1(P)$.

Corollary 6.3 (Input-independence). *The Betti numbers $\beta_k(P)$ are independent of the specific input: they depend only on the topological structure of the reachable state-transition graph.*

Example 6.4 (Merge sort). Merge sort on an array of n elements has a balanced recursion tree as its state-transition graph: no cycles, so $\text{HF}_1 = 0$ and $\beta_0 = 1$. Any input of the same length produces the same (tree) state-transition graph.

7 Application: Deadlock Detection

7.1 Deadlock as a Topological Phenomenon

A program is *deadlocked* at state σ if it reaches σ and cannot make progress: all enabled transitions lead back into a cycle from which halting states are unreachable.

Proposition 7.1 (Deadlock implies cycles). *If P can deadlock or livelock, then $G(P)$ contains at least one directed cycle.*

Proof. Since the state space is finite, any infinite non-halting execution must revisit some state σ' , producing a cycle $\sigma' \rightarrow \dots \rightarrow \sigma'$. $\square$

Theorem 7.2 (Deadlock detection via Floer homology). $\text{HF}_1(P) \neq 0$ implies $G(P)$ contains a non-contractible directed cycle (necessary condition for deadlock or livelock):

$$\text{HF}_1(P) \neq 0 \implies \exists \text{ a non-contractible cycle in } G(P).$$

Proof. By theorem 6.1, $\text{HF}_1(P) \cong \text{HM}_1(G(P))$. A non-zero element $[c] \in \text{HM}_1(G(P))$ is a 1-cycle c with $\partial_1(c) = 0$ not the boundary of any 2-chain. The condition $\partial_1(c) = 0$ implies every vertex is visited an even number of times; hence at least one vertex is visited more than once, yielding a directed cycle in $G(P)$. $\square$

Remark 7.3. theorem 7.2 gives only a *necessary* condition: $\text{HF}_1 \neq 0$ implies cycles exist, but the program may still terminate if every cycle eventually exits to a halting state.

7.2 Python Example and JuGeo Integration

The following illustrates a classic deadlock scenario: two threads acquiring the same two mutexes in opposite order.

Listing 1: Potential deadlock: two threads competing for two mutexes.

```

1 import threading
2
3 lock_A = threading.Lock()
4 lock_B = threading.Lock()
5
6 def thread_1():
7     with lock_A:
8         with lock_B:           # Thread 1 holds A, waits for B
9             pass

```

```

10
11 def thread_2():
12     with lock_B:
13         with lock_A:      # Thread 2 holds B, waits for A
14             pass
15
16 # Reachable states: (owner_A, owner_B) in {None, 1, 2}^2
17 # Deadlock: owner_A=1, owner_B=2 (circular wait)
18 # HF_1 detects the 4-cycle:
19 #   (N,N) -> (1,N) -> (1,2) -> (N,2) -> (N,N)

```

The reachable state-transition graph contains the 4-cycle shown in the comment. By theorem 7.2, $\text{HF}_1 \neq 0$, and JU GEO reports the deadlock candidate as a structured RUNTIME-tier obstruction:

Listing 2: JuGeo Floer analysis output for the mutex example.

```

JuGeo Floer Analysis: thread_1 / thread_2
Reachable states   : 4
Transitions       : 4
HF_0 rank (beta_0): 1 (one connected component)
HF_1 rank (beta_1): 1 (one independent cycle -- deadlock candidate)

Obstruction [RUNTIME tier]:
Coordinate : HF_1 generator
Violation  : Non-contractible 1-cycle in state graph
Cycle      : (N,N)->[T1 acq A]->(1,N)->[T2 acq B]->(1,2)
             ->[T2 rel B]->(N,2)->[T1 rel A]->(N,N)
Repair     : Impose consistent lock-acquisition ordering

```

8 Integration with the Judgment Geometry Framework

8.1 Floer Invariants as Trust-Annotated Judgments

Floer homology groups are computed from the reachable state-transition graph, placing them at the RUNTIME trust tier of the JU GEO framework.

Definition 8.1 (Floer judgment). For program P and degree $k \in \{0, 1\}$, the k -th Floer judgment is the JU GEO 8-tuple:

$$\mathcal{J}_P^{\text{HF}_k} = (\text{HF}_k, \beta_k(P) = r, \text{CF}_k(P), [\text{transition graph}], \emptyset, \mathcal{B}_k, \text{RUNTIME}, G(P)),$$

where $\mathcal{B}_k$ is non-empty iff $r > 0$ for $k = 1$ (deadlock candidate).

8.2 The VERIFY Semantic Move

Computing $\text{HF}_k(P)$ is an instance of the VERIFY semantic move:

- (i) **Extract:** Build $G(P) = (V, E)$ by executing P on a representative sample of inputs or by static CFG analysis.
- (ii) **Compute:** Evaluate the boundary matrix $M \in \mathbb{Z}_2^{|V| \times |E|}$ and compute $\ker M$ and $\text{im } M^T$ via Gaussian elimination over $\mathbb{Z}_2$.

- (iii) **Report:** Issue the Floer judgment at trust tier **RUNTIME**, or escalate to **ORACLE** if the state space exceeds a configurable bound.

Proposition 8.2 (Complexity). β_0, β_1 can be computed in $O(|V| \cdot |E|)$ time via sparse Gaussian elimination over $\mathbb{Z}_2$ (each column of the incidence matrix has exactly two non-zero entries).

8.3 Connection to Sheaf Cohomology

The sheaf-theoretic approach of Papers 3–6 computes Čech cohomology $H^k(\mathcal{S}, \mathcal{F})$. The Universal Coefficient Theorem over $\mathbb{Z}_2$ gives:

$$\mathrm{HF}_k(\mathbf{P}; \mathbb{Z}_2) \cong H^k(\mathcal{S}_{\mathbf{P}}, \mathcal{F}_{\mathbf{P}}; \mathbb{Z}_2).$$

Cohomological descent obstructions (obstructions to gluing local sections) and Floer-homological obstructions (non-contractible cycles) are thus two views of the same underlying geometry.

9 Related Work

Floer theory. Floer homology was introduced in [1] and extended to many settings; see [2] for a survey. The PSS isomorphism $\mathrm{HF} \cong \mathrm{HM}$ is due to Piunikhin–Salamon–Schwarz [3].

Topological program analysis. Algebraic topology has been applied to programs via persistent homology [4] and homotopy type theory [5]. Our approach applies the full Floer chain complex to the execution trace space, not a simpler point-cloud filtration.

Deadlock detection. Classical algorithms include resource allocation graph analysis [6] and Petri net coverability [7]. Our topological invariant HF_1 detects necessary conditions for deadlock and is composable with the JUGE trust algebra.

Model checking. LTL/CTL model checking [8] detects liveness violations via cycle detection. Our approach is less precise (any cycle vs. a specific formula violation) but more lightweight and compositional, producing a trust-annotated obstruction.

Judgment Geometry series. This paper builds on the semantic sites of Paper 1 [9], the Čech obstruction machinery of Papers 3–4 [10, 11], and the trust algebra of Paper 5 [12].

10 Conclusion

We have developed Floer homology as a tool for program analysis, identifying execution traces with critical points of an action functional and proving $\partial^2 = 0$ for the associated chain complex. The main theorem— $\mathrm{HF}_*(\mathbf{P}) \cong \mathrm{HM}_*(G(\mathbf{P}))$ —reduces Floer homology to graph homology: β_0 counts connected components and β_1 counts independent cycles of the reachable state space.

The principal application is deadlock detection: $\beta_1 > 0$ is a computationally checkable necessary condition for deadlock, which JUGE surfaces as a **RUNTIME**-tier structured obstruction, computable in $O(|V| \cdot |E|)$ time via sparse Gaussian elimination over $\mathbb{Z}_2$.

The Floer and sheaf perspectives are unified by the Universal Coefficient Theorem: cohomological descent obstructions coincide with Floer-homological cycle obstructions over $\mathbb{Z}_2$. Higher Floer invariants—Floer K -theory, Floer cobordism—as sources of progressively finer program invariants remain for future work.

References

- [1] A. Floer. Morse theory for Lagrangian intersections. *J. Differential Geometry*, 28(3):513–547, 1988.
- [2] D. Salamon. Lectures on Floer homology. In *Symplectic Geometry and Topology*, pages 143–229. AMS, 1999.
- [3] S. Piunikhin, D. Salamon, M. Schwarz. Symplectic Floer-Donaldson theory and quantum cohomology. In *Contact and Symplectic Geometry*, pages 171–200. Cambridge Univ. Press, 1996.
- [4] H. Edelsbrunner, D. Letscher, A. Zomorodian. Topological persistence and simplification. *Discrete & Computational Geometry*, 28(4):511–533, 2002.
- [5] The Univalent Foundations Program. *Homotopy Type Theory*. Institute for Advanced Study, 2013.
- [6] E. G. Coffman, M. J. Elphick, A. Shoshani. System deadlocks. *ACM Computing Surveys*, 3(2):67–78, 1971.
- [7] J. Esparza, M. Nielsen. Decidability issues for Petri nets. *Bulletin of the EATCS*, 52:245–262, 1994.
- [8] E. M. Clarke, E. A. Emerson. Design and synthesis of synchronization skeletons. *Workshop on Logic of Programs*, LNCS 131, pages 52–71. Springer, 1981.
- [9] H. Young. Semantic sites for program verification. *Judgment Geometry* Paper 1, 2024.
- [10] H. Young. Descent obstructions in program verification. *Judgment Geometry* Paper 3, 2024.
- [11] H. Young. The trust ordered algebra. *Judgment Geometry* Paper 4, 2024.
- [12] H. Young. SMT fragment dispatch. *Judgment Geometry* Paper 5, 2024.